

Workflow Change Log

Change Log		
Date	**Summary Of Changes**	**Author**
Aug 12, 2026	First release	*Antonio N.*

Reviewer Golden Workflow

This document describes the golden workflow every reviewer MUST follow when working on tasks in this project. Follow the steps in order.

Step 1 — Set up the sandbox files (CRITICAL)

> ⚠️ **This step is the most important part of the workflow. Do NOT skip or reorder any part of it. If the sandbox is not set up correctly, the rest of the task will be invalid.**

The sandbox must be populated with the correct task files before you do anything else. Follow every sub-step in order:

1a. Download the files below the sandbox

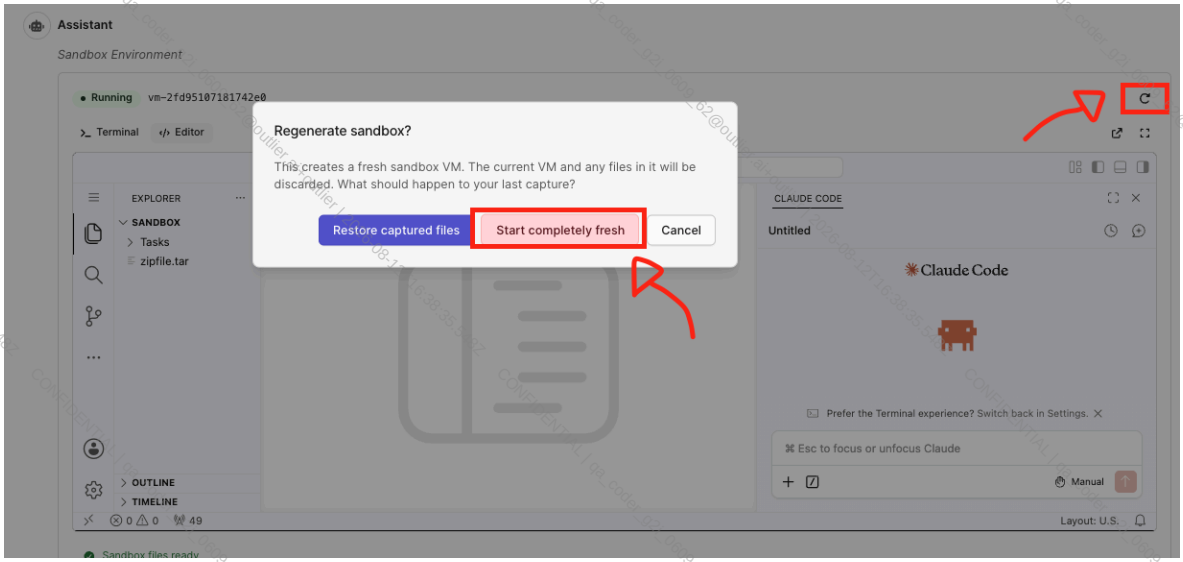
Download **every** file listed in the **Captured files** section below the sandbox. These are the files you will be working with:



1b. Regenerate the sandbox completely fresh

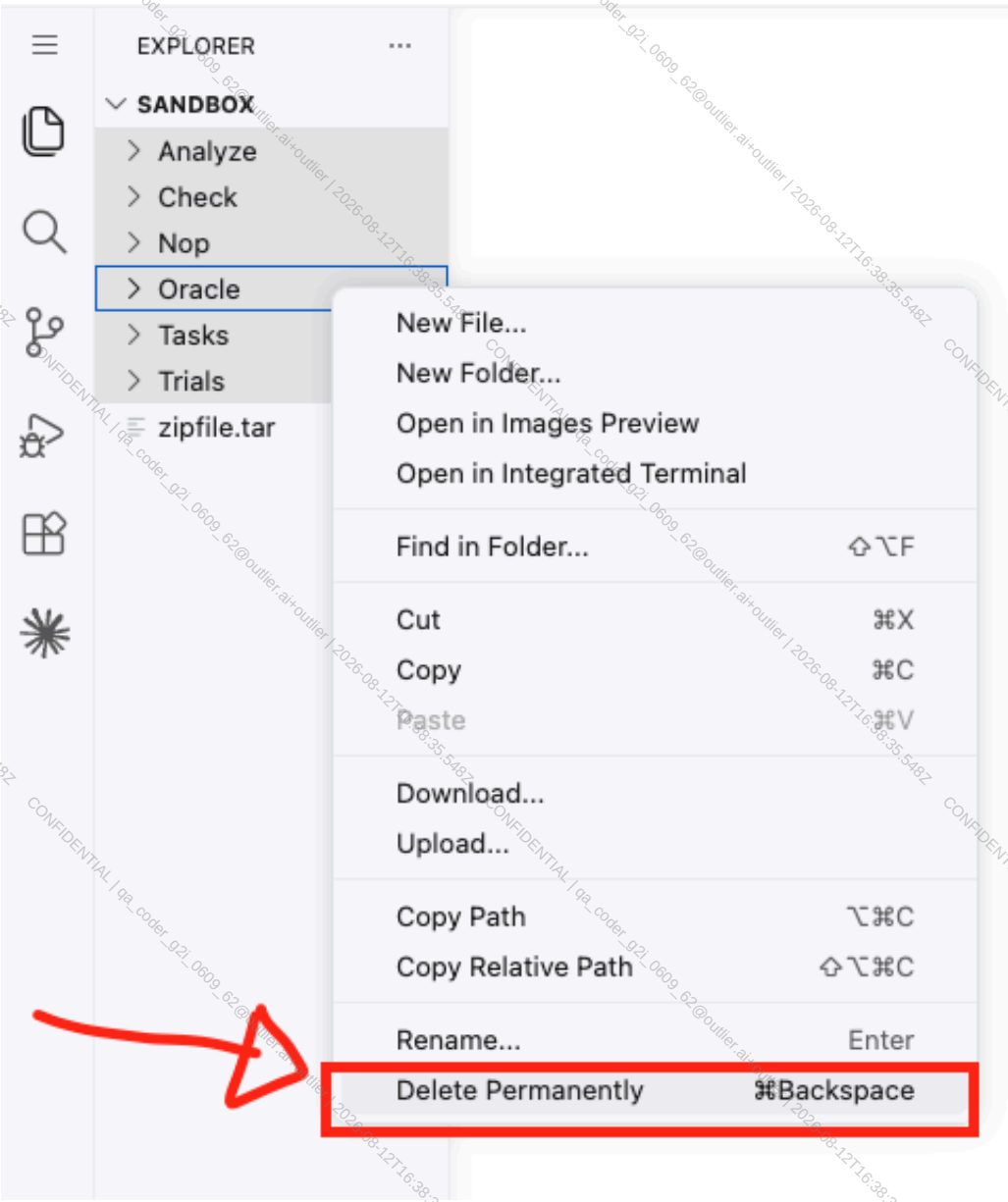
After you have downloaded the captured files, regenerate the sandbox from scratch. Click the **regenerate** icon in the top-right of the sandbox, and in the **Regenerate sandbox?** dialog, choose **Start completely fresh**.

> ⚠️ **This creates a brand-new sandbox VM and discards the current VM and all of its files.** Make sure you have already downloaded the captured files in step 1a before doing this — anything not downloaded can be lost.
>
> ❌ **Do NOT choose "Restore captured files".** That button does not consistently capture the previously submitted files into the sandbox, so you will not get the latest work done by the attempter. If you rely on it, you will end up reviewing incomplete or outdated work and **unfairly evaluate the task**. Always choose **Start completely fresh** and bring in the files you downloaded in step 1a instead.



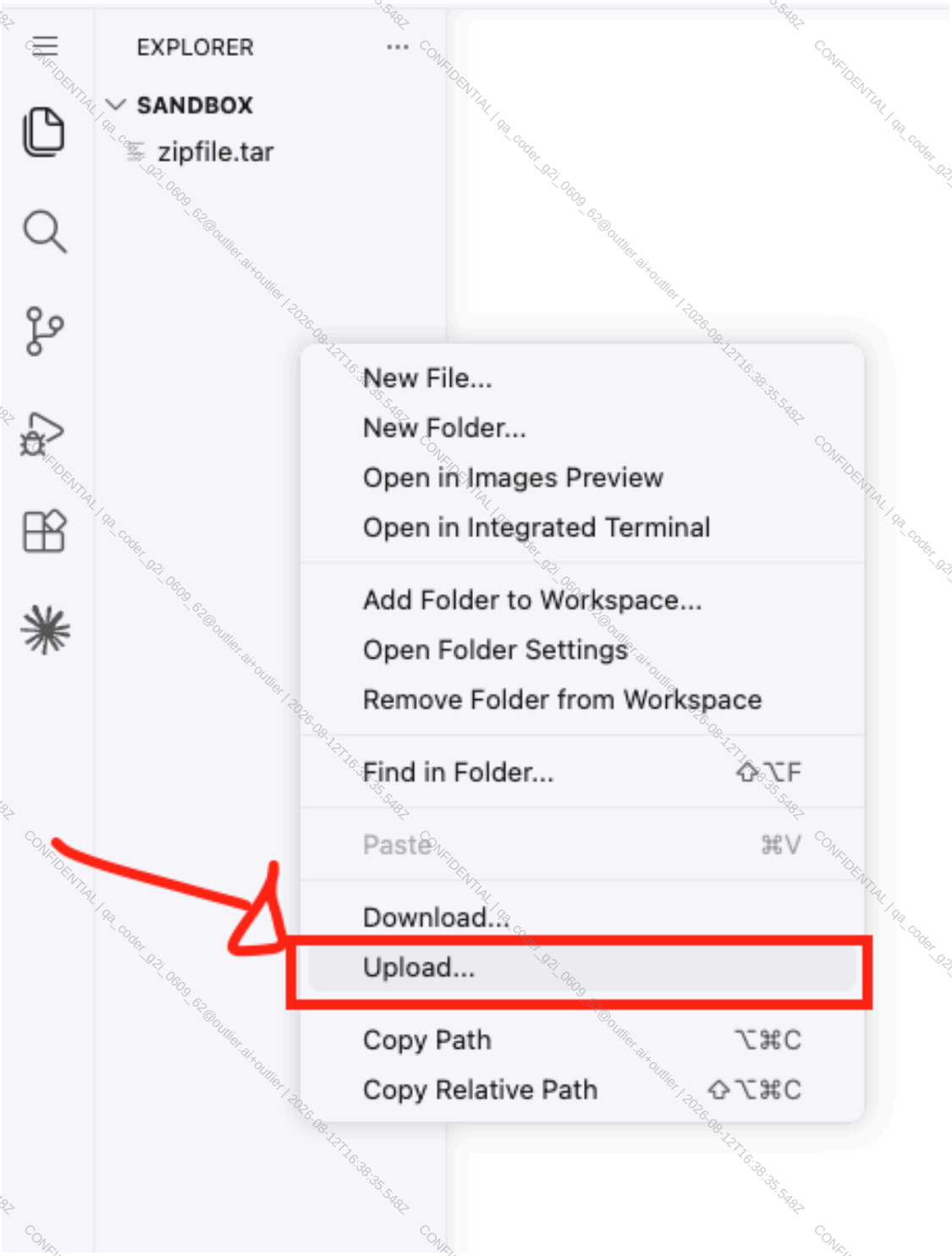
1c. Delete the existing files in the sandbox

Delete all of the files currently in the sandbox. Right-click each item in the Explorer and choose **Delete Permanently**.



1d. Upload the downloaded files into the sandbox

Right-click the **whitespace** in the Explorer tab and choose **Upload…**, then upload the files you downloaded in step 1a.

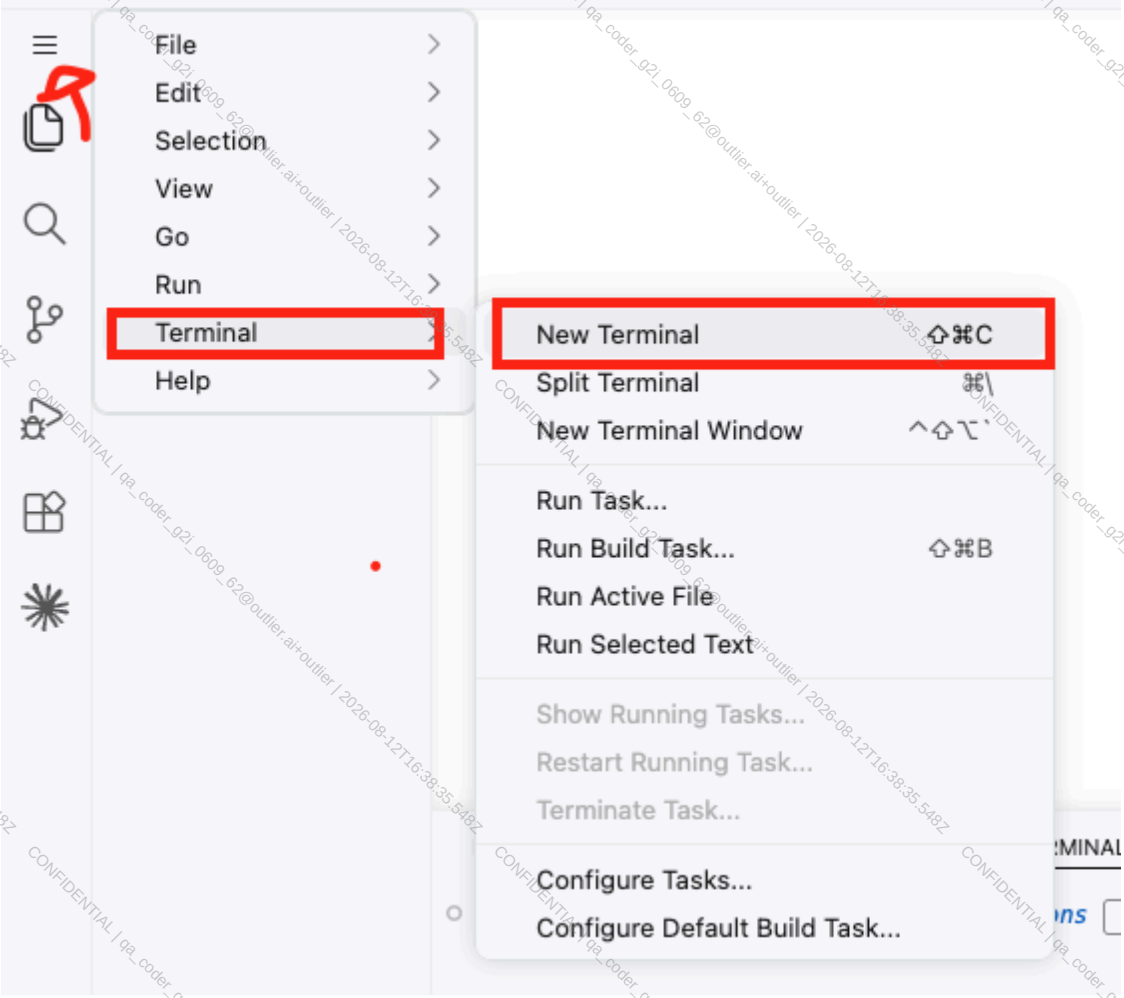


📦 **You MUST upload the files as `.tar.gz`** — do not extract them on your machine first. They need to be uploaded in `.tar.gz` form because they will be extracted inside the sandbox in the next sub-step.

CONFIDENTIAL | qa_coder_g2i_0609_62@outlier.ai+outlier | 2026-08-12T16:38:35.548Z

1e. Open a terminal in the sandbox

Open a new terminal in the sandbox: from the menu, choose **Terminal → New Terminal**.



1f. Extract the uploaded files

Run the following command to extract every `.tar.gz` archive you uploaded:

```
for file in *.tar.gz; do tar -xzvf "$file"; done
```

Once all archives have been extracted, the sandbox is fully set up and you can proceed to Step 2.

Step 2 — Run the quality check with Claude

With the sandbox files extracted, run the automated quality check.

2a. Open a Claude chat

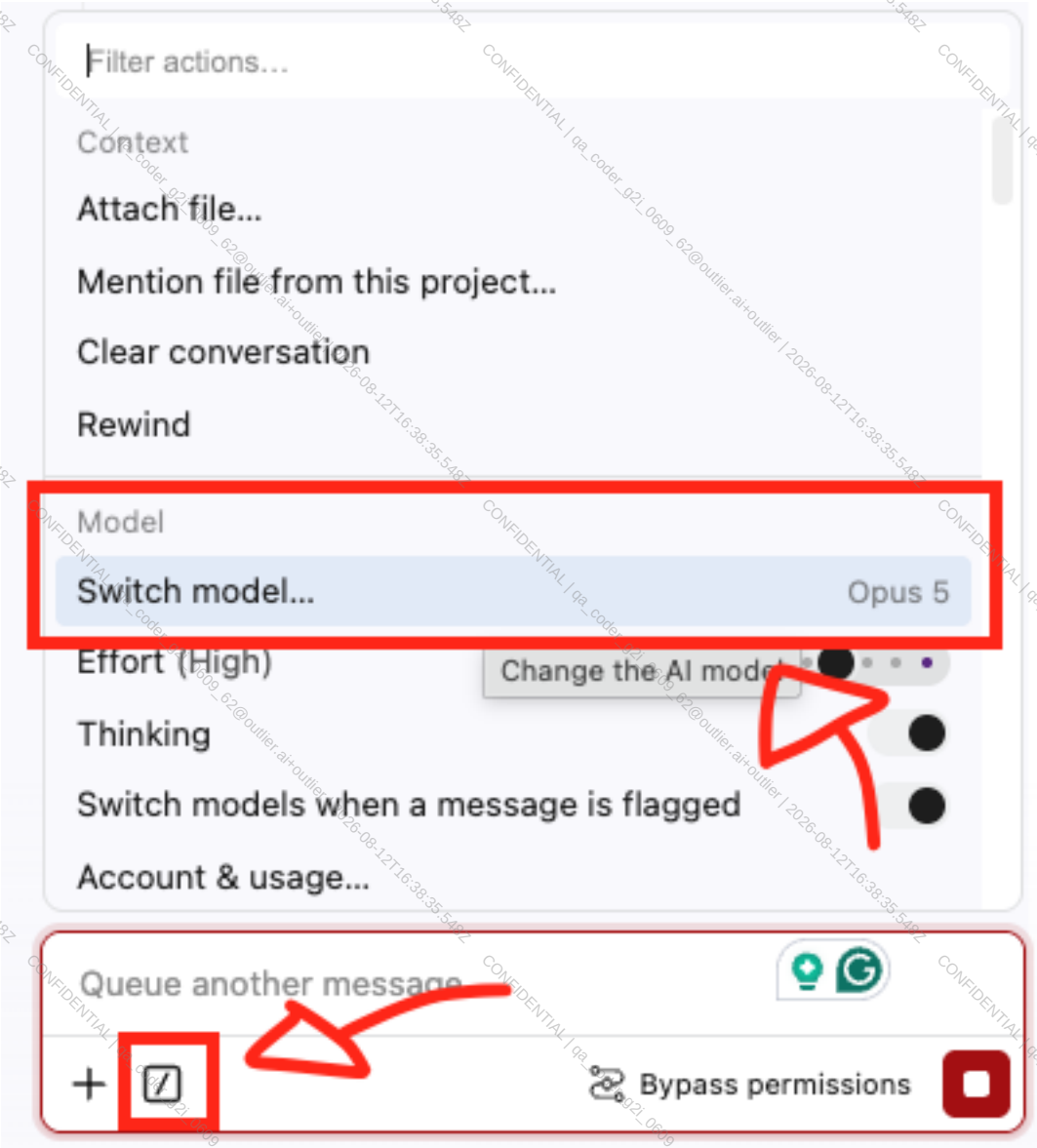
Open a new Claude Code chat.



2b. Switch the model to Sonnet 5

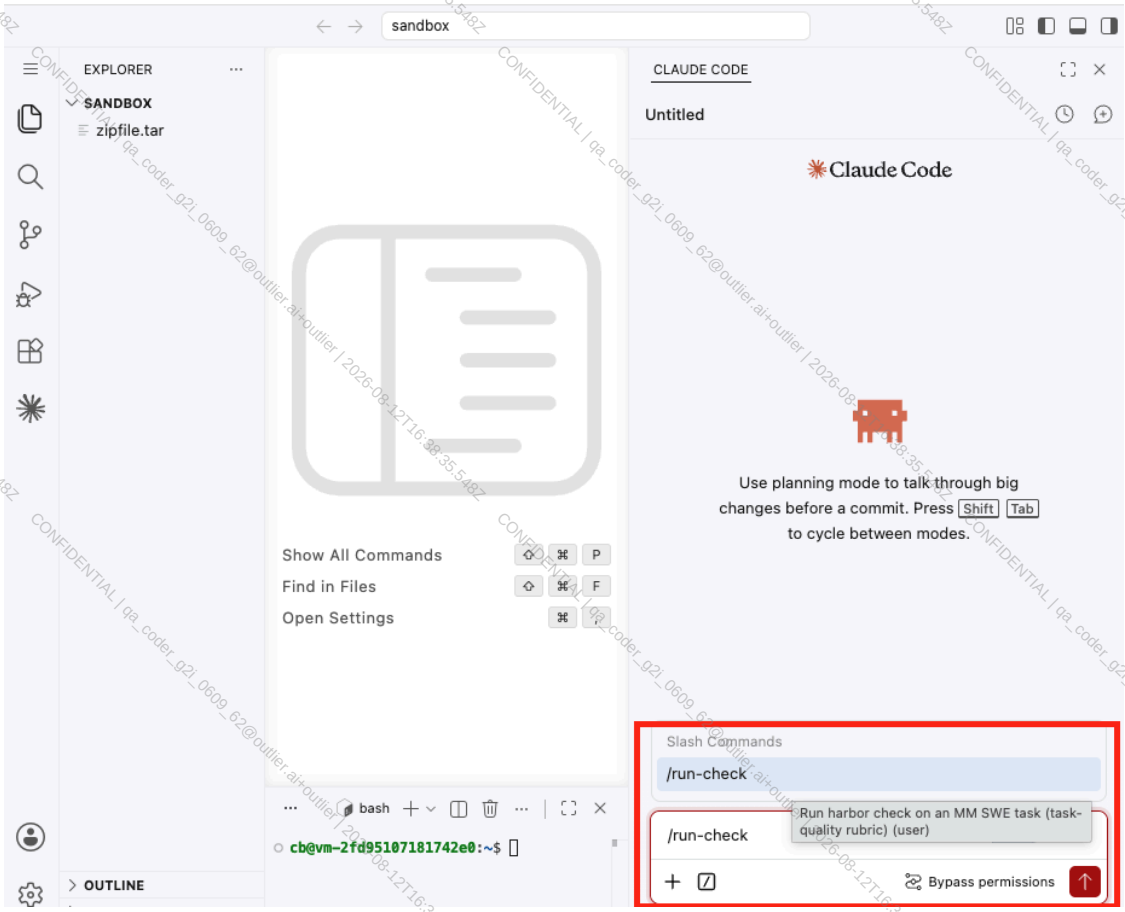
Open the model menu (/ → **Switch model…**) and select **Sonnet 5**.

CONFIDENTIAL | qa_coder_g2i_0609_62@outlier.ai+outlier | 2026-08-12T16:38:35.548Z



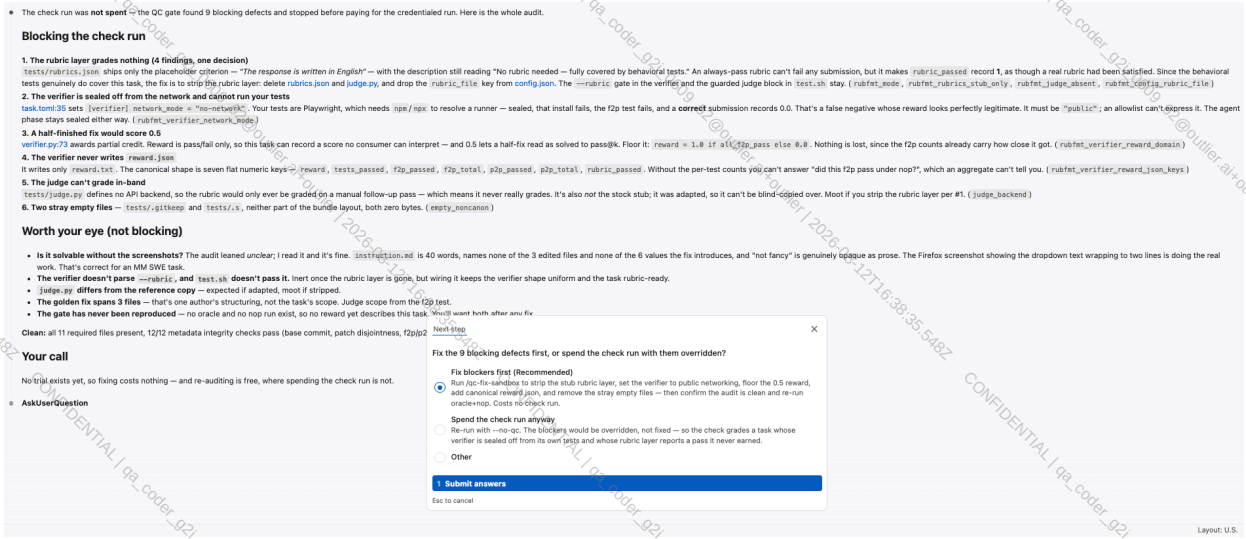
2c. Run the `/run-check` command

Type and run the `/run-check` slash command. This runs the harbor check on the task against the task-quality rubric.



2d. Review the output and fix the blockers

`/run-check` will produce an output like the one below:

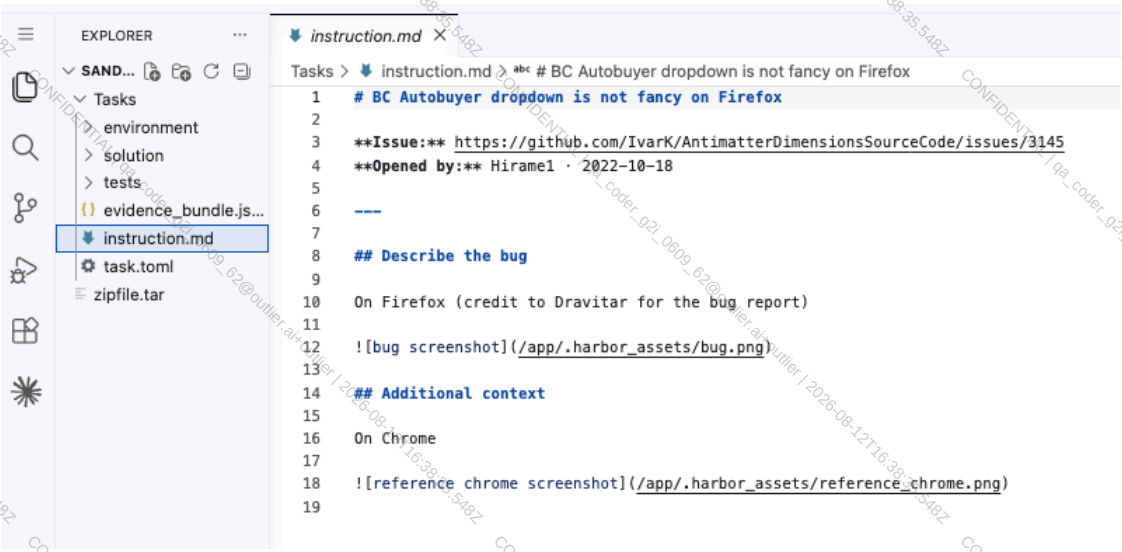


Given this output:

- **The blocking issues are always correct**, so you will **always** prompt the model to **"Fix blockers first"**.
- **Before** you let the agent fix the blockers, read through the **non-blocking** issues (e.g., the **"Worth your eye"** section) and decide whether you agree or disagree with each of them.
- After reviewing the non-blocking issues, allow the agent to **fix the blockers**.

Step 3 — Check whether the instruction is image-dependent

Open the task's `instruction.md` and read it carefully.



The goal here is to verify that the prompt is **image-dependent** — the model should **need the screenshots** to understand and solve the issue. If the prompt is so detailed that the model can solve the issue from the written text alone (without looking at the images), then the instruction is **incorrect** and must be trimmed down.

Example: how the screenshot above could be improved

The `instruction.md` shown in the screenshot is too detailed. It should be improved by:

- **Deleting the issue URL** (the `**Issue:** https://...` line) and the **`**Opened by:**`** statement.
- **Removing the title** "BC Autobuyer dropdown is not fancy on Firefox" — it gives away too much information. Just by looking at the two screenshots, a human would be able to understand the issue.
- **Keeping only the following written text**, followed by the two screenshots:

  On Chrome, the display is better aligned.

reference chrome screenshot

reference chrome screenshot

bug screenshot

bug screenshot

This leaves the reviewer/model relying on the screenshots to understand the difference, which is exactly what an image-dependent task should do.

Step 4 — Check whether the verifiers are truly implementation-agnostic

Review the task's verifiers:

- `tests/test.patch`
- `tests/rubrics.json` (only if your task has rubrics)

The goal is to confirm the verifiers are **implementation-agnostic**. A verifier is **implementation-specific** when it would **reject a distinct but valid solution** to `instruction.md` and only accepts a solution that is **identical to the golden solution** (`solution/fix.patch`). Verifiers should never behave this way — any correct approach to the issue should pass.

What to do

- If the tests are **overly specific**, you **MUST** rewrite them to be **implementation-agnostic** so that any valid solution passes.
- If you **cannot** make the tests properly implementation-agnostic — because there could be **dozens of valid solutions** to the problem (e.g., centering a div) — you **MUST create rubrics** (`tests/rubrics.json`) to evaluate the solution instead.

Step 5 — Rerun the trial

Rerun the trial from the terminal with the following command:

```
bash ~/.claude/skills/mmswe-runner/mm_run.sh /home/sandbox/Tasks trial
```

If the model **keeps passing** the trial, one of the following is true:

- Your **verifiers** (`tests` / `rubrics`) are **too lenient** and are allowing incorrect solutions, **or**
- Your **`instruction.md`** is **not hard enough**.

If neither of those is the case, you must make the task **harder** by introducing **more visual bugs** to the codebase, following the **Hardening Deep Dive** document.